Automating the Management of Software Systems

WIP - DRAFT

1 The Design

A design is a closed semantic world. It contains participants whose state is transformed through the design's unambiguous behaviors. When the environment interacts with the design, it introduces a change to the state of the world. This state transition may enable new behaviors, which the design deterministically resolves in response to the updated state, producing further state changes. When resolving a state, the design may either exhibit behavior directly or deterministically select which behaviors are enabled as a reaction to the state. The design continues reacting to each resulting state transition until no further behavior is enabled and the world reaches a stable semantic state. This chain of resolutions constitutes the structure of the design itself and provides the environment a fully specified path through it.

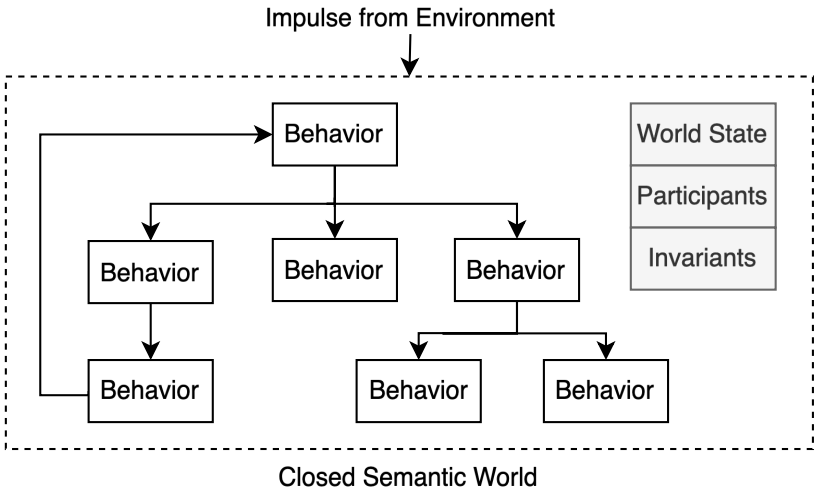

Figure 1: The Design

1.1 Semantic Invalidity

The design is said to be in a semantically invalid state when one of its intentions can't be realized as a result of that state. When the environment provides an input into this closed semantic world, in order to continue successfully realizing its intentions, the design must prevent semantically invalid states from persisting in its world.

In order for the closed semantic world to participate in reality, it must be implemented on a substrate. When realized on a substrate, reality can refute the assumptions of the closed semantic world and the design fails because its intentions can't be realized. As a result, the design must learn new semantics to continue realizing its intentions. So in a closed semantic world, a failure can be understood as an assumption made by the design that reality refutes and I will be using the world failure in this context for the rest of the document. When the implementation on a substrate halts because semantic invalidity was allowed to persist, I will refer to this as a mechanical failure.

To prevent semantically invalid states from persisting in its reality, semantic invariants act as markers that identify the semantically invalid state and predict the failure of downstream behaviors. This establishes an invariant path from the semantically invalid state to the intention that can't be realized and in order to restore semantic validity, the design must provide a semantically valid path to resolve the state.

1.2 Narrative of the World

Another way of describing a closed semantic world is through the narratives that are established by the design. The narratives are constructed using the behaviors and how it unambiguously modifies the state of the worlds participants. When looked at in this way, a semantically invalid path is a narrative that will not succeed in reality. When the design learns new semantics, it absorbs the awareness of the invalid narrative into its domain knowledge. Another way to say this is that the there exists a narrative in the world that is aware of the invalid narrative and it prevents any impulse from the environment from reaching it.

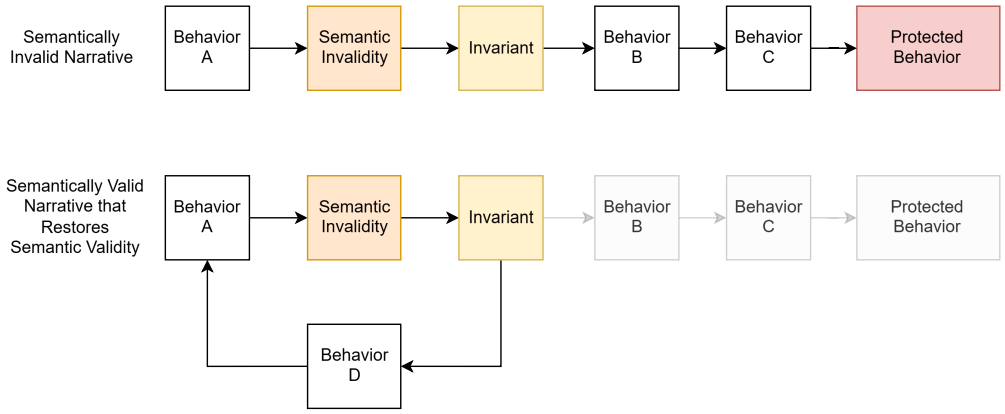

Figure 2: Semantically Invalid Narrative and Semantically Valid Narrative

1.3 Meaning of the World

As part of the design, each behavior is has an unambiguous meaning, it unambiguously transforms the world state. When a behaviors meaning is established as a composition of behaviors, it is called a composite behavior. The composite behaviors meaning is established by the semantics of the behaviors that compose it. Consider the following example:

- Place Book on The Shelf
  - Get Book
  - Get Book Name
  - Get First Letter of Book Name
  - Add Book to Shelf

In this example, the meaning of the behavior placeBookOnShelf is composed by the specified behaviors. This is no different than how words work as part of human communcation, the word lion is a compression of the meaning of a lion, this includes the apperance, sound etc. A closed semantic world unambiguously establishes the meaning within its world. Through the composition of behaviors, each level of meaning establishes a coherent world.

1.3.1 Shared Meaning

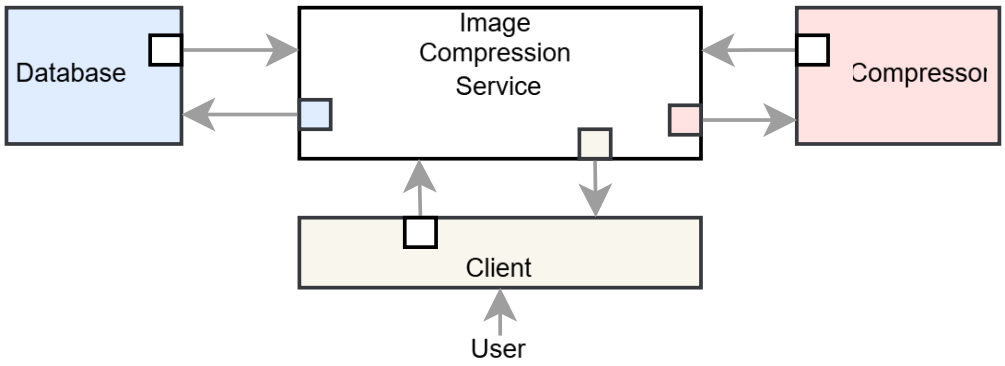

Figure 3: Semantic Comaptibility through Shared Meaning

When two designs interact with each other and have shared meaning, they form a larger semantic world through semantically compatible interactions. In this way, each part of a larger distributed system can be designed and assembled ino a larger closed semantic world.

Figure 3 demonstrates this using a distributed image compression system. For example, when the database is designed, the meaning of the database user is established. When the image compression service interacts with

the database, it assumes the role of the databse user, thus establishing shared meaning with the database.

The next section describes how the world can be reasoned about through its computable semantic model. The same principles described are applicable to larger closed semantic worlds construtured through semantically compatible interactions.

1.4 Reasoning about the World

Since the design is a closed semantic world, it is possible to reason about the world. For example, the invariants placed on a behavior can be used to reason about where to place the semantic invariants. This is possible because the narrative of the world unambiguously identifies where the participants enter a semantically invalid state and will inevitably reach the behavior. Furthermore, it is possible to identify the path that should be established to restore the semantically valid state.

This form of reasoning can be used to perform many meaningful actions on the world and I will list a few below:

- The ability to place invariants and resolve the semantically invalid states is failure prediction and resolution within the closed semantic world.
- The ability to walk down semantically invalid narrative paths and identify if the design selects a semantically valid narrative is testing the world.
- If all the invariants are tested but the design still fails, the failure is automatically diagnosed because it can only mean that new semantics must be learnt.

Together, these abilities establish debugging as a process of reasoning about the world given that the designs meaning is faithfully represented by its implementation. The kinds of reasoning that are possible are only limited by the meaning established by the design.

If the closed semantic world was established in an unambiguous and structured form, then an engine would be able to automatically reason about the world. In the next section, I will discuss the implications of this capability.

2 Semantic Reasoning Engine

As the previous section established, the design is a structure that can be reasoned about through the meaning that it establishes. As such, if the design is established in a structured and unambiguous form, an engine can automatically perform the semantic reasoning.

To make this possible, the design must be written in a Design Abstraction Language (DAL). The DAL is a declarative language that is used fully establish the closed semantic world. The scope of the DAL includes all the meaning needed to fully define the world (behvaiors, participants, attributes, invariants, etc).

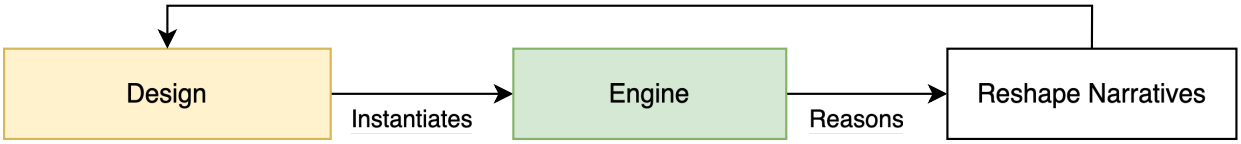

Figure 4: Automated Reasoning to Reshape Narrative

An engine can use the DAL definition to automatically reason about the world. It can use the meaning of the design to enforce its own definition of correctness. The ability of the engine to reason about the world is limited by the meaning that has been established. When the design participats in reality, a failure can be seen as a meaningful quesiton that can't be answered by the engine.

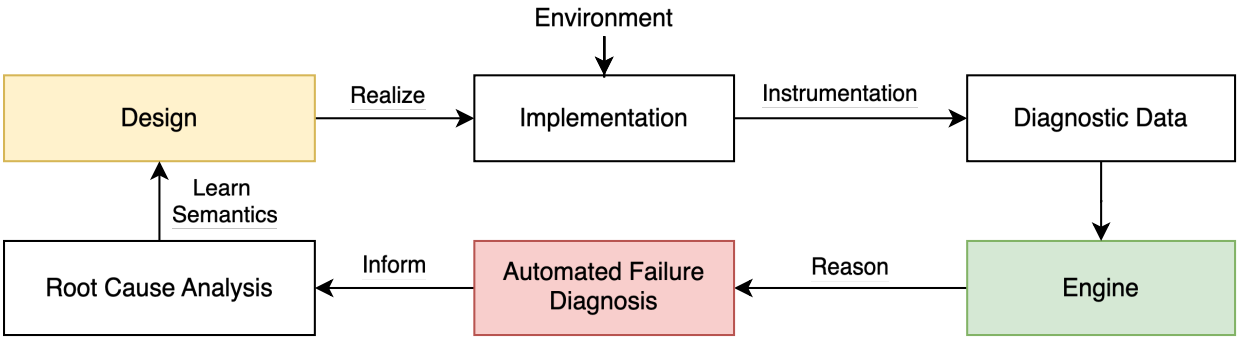

Figure 5: Ideal learning loop by reasoning about execution using design meaning

By reasoning about the world, the engine can eliminate any semantically invalid narratives. This means that by reasoning about the execution of the design to identify semantically invalid states, the failure can be automatically diagnosed. This is possible because that the engine can use the meaning of the constructed world to automatically diagnose the failure in the execution and identify where the semantics of the world has to expand through root cause analysis. Through this process, the engine will learn how to answer the meaningful question and eliminate the failure.

To do this, the following steps are necessary:

- The implementation and its execution must represent the design semantics faithfully.
- The execution must be losslessly preserved for the reasoning to be deterministic.

If the semantic gap between the implementation and the design is eliminated and the execution is losslessly preserved, it would enable a determinstic learning loop. More importantly, as a result of this, any meaningful question about the world can be automatically answered or an automatic process would collect the necessary diagnostic information needed to learn how to answer the question.

In the following sections, I will explain how a Computable Semantic Model (CSM) can be used to eliminate the semantic gap between the design and the implementation. Then I will explain how domain specific compression can be used to losslessly preserve all the narratives experienced in the world so the engine can deterministically reason about it and finally I will explain how all of this works to fully automate systems management.

3 Computable Semantic Model

In order for the engine to reason about the execution of the design, the semantic gap between the design and the implementation must be eliminated. If this is not done, then the engine cannot deterministically reason about the world because there will be no way to know if the semantics need to expand or if the implementation doesn't realize the meaning of the design.

If the implementation does realize the meaning of the design faithfully, then all the automation that is enabled by the engine reasoning about the design can be leveraged. To make this possible, the design must be established as a Computable Semantic Model (CSM) that is built from Computable Semantic Primitives (CSP) and I will describe how in the following sections.

3.1 Implementing a Computable Semantic Model

A CSM is built by establishing the implementation of the designs meaning enabling the implementation of the design to be directly synthesized into an executable form. The design establishes the first behavior it exhibits and then the design reacts to each impulse from the environment until it reaches a semantically stable state.

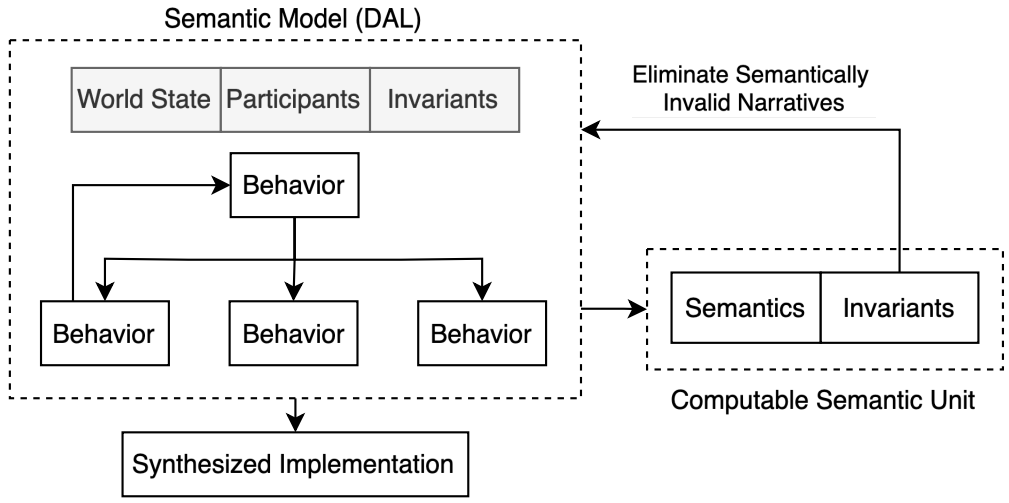

Figure 6: Computable Semantic Model

However, there is still the possibility that the provided implementation doesn't actually realize the meaning of the design, in which case, the engine cannot reason about the world in a reliable way. Moreover, the engine can only reason about the world based on the meaning that was specified. So unless the design makes all the releveant semantics available, the engine will be unable to determine the worlds correctness. So the engine will know that there is a failure and it won't be able to explain why until it learns the missing semantics.

In the next section, I will describe how designs built from Computable Semantic Primitives addresses both of these problems.

3.2 Computable Semantic Primitives

Computable Semantic Primitives (CSP's) are primitive computable semantic units that can be used to construct the meaning of the semantic world. Since the world is constructed from meaningful atoms that have uambiguous invariants, it can be reasoned about completely to eliminate all semantically invalid narratives. Since the primitive units can be synthesized into an implementation by definition, this transforms the DAL into a declarative language that establishes the meaning of a closed semantic world and synthesizes the meaning of that world as an implementation, effectively making the design itself executable.

As previous sections mentioned, the engines ability to reason about the execution of the world is dependent on there being no semantic gap between the design and the implementation. Using CSP's solves this problem elagantly in multiple ways:

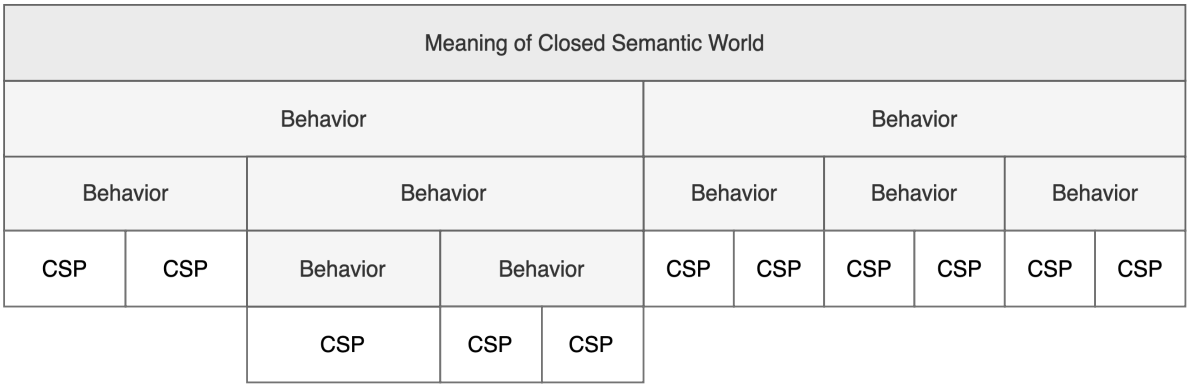

Figure 7: Computable Semantic Model constructed from Computable Semantic Primitives

1. It allows the engine to fully reason about the constructed world to eliminate any semantically invalid narratives.
2. It allows the engine to reason about the execution to automatically diagnose any failures and motivate the necessary root cause analysis to learn new semantics.

Now that the constructed model and the execution have the same meaning, the engine can automatically identify where the semantics need to expand because reality will refute the designs assumptions through failures.

However, this automation will still not be possible unless the engine actually has a complete account of the execution to diagnose the failure. In the next section, I will talk about the information that needs to be preseved to make this possible.

3.3 Automated Instrumentation

Since the design is an unambiguous structure, it isn't necessarsy to preserve all the execution information to be able to replay it. The design itself can identify the information that can't be deterministically computed by the design. This means that the synthesized implementation can be automaticaly synthesized to only include the non-deterministic information.

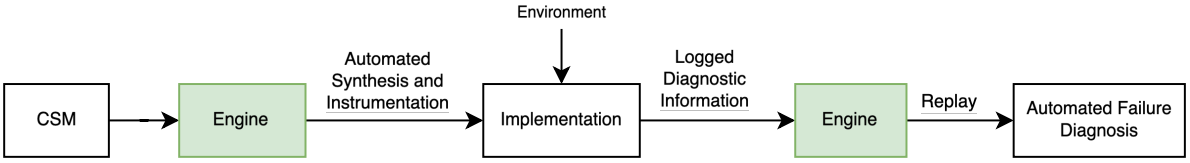

Figure 8: Automated Instrumentation and Synthesis

As Figure 8 illustrates, the engine can use the unambiguous specification of the CSM to identify the necessary instrumentation. The information produced by the execution is then fed back into the engine where the failure is automatically diagnosed. In this way, the ways in which reality refutes the assumptions made by the design can be unambiguously identified and the learning process can be triggered.

However, to truly make this an automated process, all the diagnostic data needs to be preserved at scale. Any missing data would introduce speculation, so there is a need for a solution address the practical problem while minimizing cost. In the next section, I will describe how this can be addressed by using the unambiguous domain structure of the data to apply domain specific compression.

4 Domain Specific Compression

In order to preserve all the diagnostic data necessary for the engine to deterministically reason about the world, the unambiguous domain structure of the CSM can be leveraged to apply Domain Specific Compression (DSC). This technique leverages the known domain structure of the data to optimally compress the data by exploiting the unambiguous nature of the data.

4.1 Compressed Log Processor

Practically, this can be achieved using a tool named Compressed Log Processor (CLP). It is an open source tool that can apply DSC and search the compressed data without decompression. More importantly, it has proven that DSC is practical at a petabyte scale. Currently, CLP compresses structured and unstructured logs by exploiting their known domain structure. However, in its current form, the domain structure of the variables is not know, so CLP is not maximizing compression it can achieve.

Figure 9: CLP Logo

Using the CSM, CLP can be transformed into a DSC engine. CLP's gains come from a lack of ambiguity and the CSM works to eliminate ambiguity entirely in its domain, thus CLP's compression will improve significantly through this process. More importantly, since the data exists as part of a structure with unambiguous meaning, the compression will improve further by exploiting the repetitive and predicable nature of programs.

When the engine replays the execution using the collected diagnostic to automatically diagnose the failures, the narratives that are played out in the semantic world can be preserved by CLP at minimal cost. This means that the engine will never have to re-compute the same transformation twice. Instead, it can leverage CLP's search without decompression feature to identify if the narrative has already been computed.

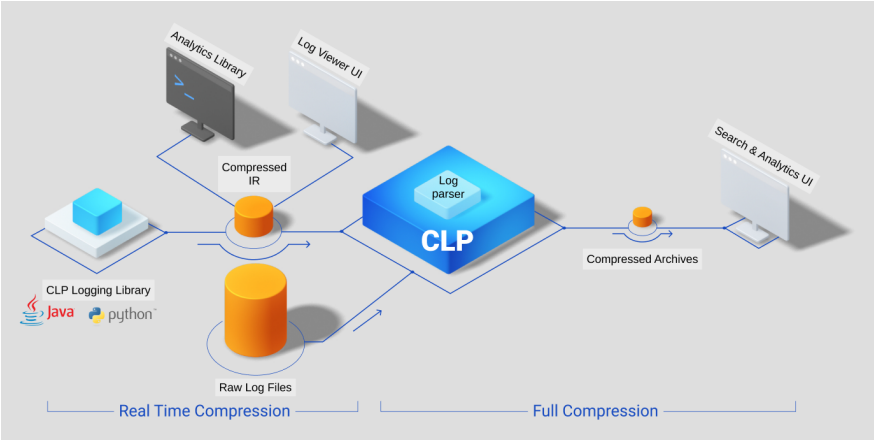

Figure 10: Overview of CLP's current solution.

This creates a very powerful ability where CLP contains the entire experiential history of the closed semantic world. When combined with the CSM, this is what allows the engine to reason about the world deterministically. CLP's search performance will also become more optimized since the both the structure and the meaning of the data are entirely unambiguous. The queries themselves are inherently semantic because the data exists as part of a closed semantic world and the queries can be meaningfully structured.

Ultimately, this will result in a fully optimized platform because the meaning of any data in the platform is inherently known. Moreover, when combined with the CSM and the engine, this results in a design learning platform that fully automates software systems management.

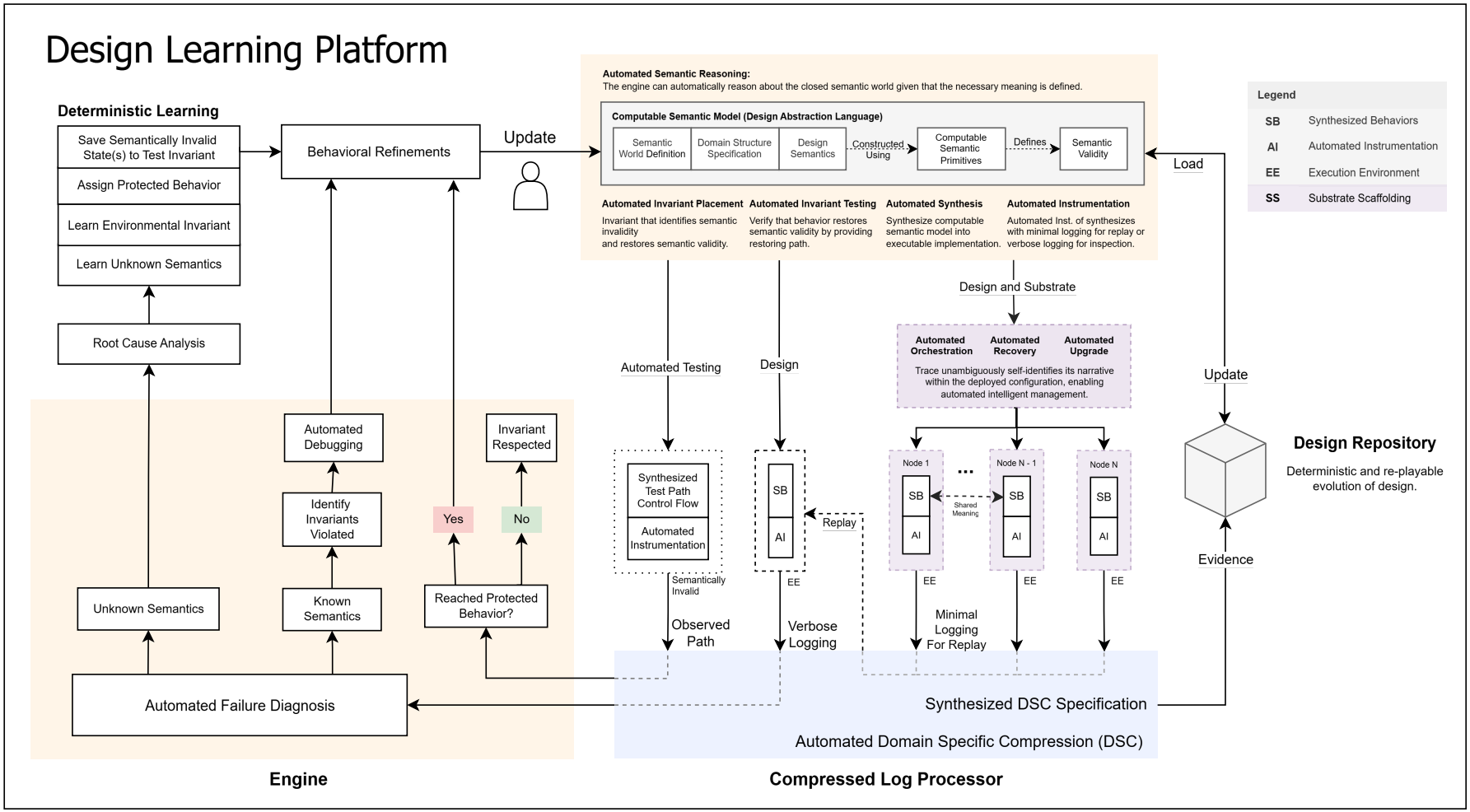

Figure 11: The Design Learning Platform

5 Design Learning Platform

In this section, I will present the Design Learning Platform (DLP) and describe how it fully automates the management of software systems. First, I will speak about how the ideas described earlier in this document enable automated systems management and then I will describe each part of the design learning platform.

6 Automated Systems Management

Systems management can be described as the process of reasoning about a system to take the appropriate action. In developing the computable semantic model, first, the structure that can be automatically reasoned about using an engine was established. Next, the semantic gap between the implementation and the CSM was eliminated using Computable Semantic Primitives (CSP), allowing the same engine to reason about the execution of the Computable Semantic Model (CSM). This enabled the engine to reason about the world, synthesize/instrument the implementation and to reason about the execution of the world. Finally, this process was enabled at scale by using Compressed Log Processor (CLP) to losslessly retain the narratives experienced by the world and to efficiently diagnose the failures from the observed narratives.

7 Conclusion

8 Extended Applications

In this section, I present some applications that can immediately benefit from this framework outside of software systems and also some future thoughts for interesting applications that I would want to explore within the world of software systems.

8.1 Universal Design Language and Digital Twins

Designs are the universal structures used across many disciplines. The ability to unambiguously establish a closed semantic world and reason about it has applications well beyond software systems. If implemented with the right stratergy, this framework has the ability to become a universal design language used in many disciplines.

Since the infromation contained in the DAL has unambiguous meaning, it can be used to establish a powerful visualization framework. This would allow engineers to visualize the closed semantic world in a meaningful way and gain insights into their design by reasoning about its meaning. More importantly, since this framework extends the design into a computable semantic model (CSM), it extends the CSM to become a digital twin of what it is modelling.

In software systems, the design establishes a closed semantic world that mirrors what partcipates in reality because the design is what defines it (this statement is even more valid with CSM's construted using CSP's). This is a unique property of software systems since they operate entirely on a stack that is symbollically defined by humans. However, in other disciplines, reality is modelled by engineers to be able to represent it accurately and predict it. In these applications, the CSM becomes the framework through which that modelling happens.

In the digital twin models, information collected by sensors is used to inform the model and refine it so that it is a more accurate representation of reality and to improve its ability to make more accurate predictions. In these cases, the domain structure of the collected data is unambiguously established by the CSM, as such, the data can be domain compressed using CLP's engine. This extends the use of CSM and CLP beyond software systems.

The applications of CSM are only limited by what can be modelled, anything that can be modelled as a closed semantic world can be reasoned about using the engine. A personal favourite of mine is to model concepts using the CSM so that the model can be used as a teaching tool. This applications exploits the fact that when humans teach, they are ultimately building a closed semantic world to convey the concept to another human mind. Using the CSM, the semantic world necessary to represent the idea can be transformed into a computable model in which every step in the process is informed and validated by the model. There are many interesting ways in which this can be used to turn teaching and learning into an unambiguous process that converges on complete understanding.

8.2 Future Exploration: Automated Development

The thoughts in this section are more future oriented but something that I would enjoy exploring. I believe that the CSM built from CSP's opens the door to interesting approaches for automating the development of software systems because the engine has the ability to reason about the world to identify its internal consistency and correctness. I think there are interesting techniques where a semantic toolkit is used to construct closed semantic worlds and expand the meaning of world.

Since failure diagnosis is automated and the design can deterministically learn new semantics through root cause analysis, I think there are applications where the design operates entirely in a virtual environment and integrates its meaning into the existing closed semantic world. In a solution like this the simulated environment itself has unambiguous meaning and when the design fails, it learns how it must modify its behavior to achieve its intentions in its environment.

It would be able to perform root cause analysis on the failure because the entire world is modelled and the meaning is unambiguous. While there are certainly many intermediate steps on the way to reaching this, I feel that when software systems are modelled in this way to establish its meaning as a computable model and they start to work together in a simulated world, solutions like this will become more realistic where designs try and fail in simulations to learn.